

Android 适配器：原生与第三方对比及选型

在Android开发中，适配器（Adapter）是连接**数据与UI组件**（如ListView、RecyclerView、Spinner等）的桥梁，负责将数据绑定到视图并管理视图的复用。本文将详细介绍Android原生适配器的分类、使用场景，以及主流的第三方替代方案，帮助你更高效地处理数据展示。

一、Android原生适配器详解

1. 适配器的核心作用

- **数据绑定**：将数据源（如List、数组、数据库）映射到UI组件的每一项视图。
- **视图复用**：通过回收复用不可见的视图（如ListView的convertView），减少内存占用和渲染开销。
- **事件处理**：可在适配器中处理子项的点击、长按等事件。

2. 原生适配器分类

（1）BaseAdapter（基础适配器）

- **适用组件**：ListView、GridView、Spinner等传统AdapterView组件。
- **核心方法**：

Code block

```
1  @Override
2  public int getCount() { return data.size(); } // 数据项总数
3  @Override
4  public Object getItem(int position) { return data.get(position); } // 获取指定位置数据
5  @Override
6  public long getItemId(int position) { return position; } // 获取项ID
7  @Override
8  public View getView(int position, View convertView, ViewGroup parent) {
9      // 创建/复用视图并绑定数据
10     if (convertView == null) {
11         convertView =
LayoutInflater.from(context).inflate(R.layout.item_layout, parent, false);
12     }
13     TextView textView = convertView.findViewById(R.id.tv_item);
14     textView.setText(data.get(position));
```

```
15         return convertView;
16     }
```

- **优化：**结合 `ViewHolder` 模式减少 `findViewById` 调用：

Code block

```
1  static class ViewHolder {
2      TextView tvItem;
3  }
4
5  @Override
6  public View getView(int position, View convertView, ViewGroup parent) {
7      ViewHolder holder;
8      if (convertView == null) {
9          convertView =
              LayoutInflater.from(context).inflate(R.layout.item_layout, parent, false);
10         holder = new ViewHolder();
11         holder.tvItem = convertView.findViewById(R.id.tv_item);
12         convertView.setTag(holder);
13     } else {
14         holder = (ViewHolder) convertView.getTag();
15     }
16     holder.tvItem.setText(data.get(position));
17     return convertView;
18 }
```

(2) ArrayAdapter (数组适配器)

- **适用场景：**简单的字符串列表展示（如Spinner、ListView）。
- **特点：**支持泛型，直接绑定数组或`List<String>`，内置默认布局（如`android.R.layout.simple_list_item_1`）。
- **示例：**

Code block

```
1  String[] data = {"Apple", "Banana", "Orange"};
2  ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
3      android.R.layout.simple_list_item_1, data);
4  listView.setAdapter(adapter);
```

(3) SimpleAdapter (简单适配器)

- **适用场景：**多类型数据（图片+文字）的简单展示，无需自定义Adapter。

- **参数：**数据源（List<Map<String, Object>>）、布局ID、数据键名数组、控件ID数组。
- **示例：**

Code block

```
1 List<Map<String, Object>> data = new ArrayList<>();
2 Map<String, Object> item = new HashMap<>();
3 item.put("icon", R.drawable.ic_apple);
4 item.put("name", "Apple");
5 data.add(item);
6
7 SimpleAdapter adapter = new SimpleAdapter(this, data,
8     R.layout.item_simple,
9     new String[]{"icon", "name"},
10    new int[]{R.id.iv_icon, R.id.tv_name});
11 listView.setAdapter(adapter);
```

(4) RecyclerView.Adapter（高性能适配器）

- **适用组件：**RecyclerView（替代ListView/GridView的新一代列表组件）。
- **核心优势：**
 - 强制使用 `ViewHolder`，优化性能。
 - 支持多布局类型（重写 `getItemViewType`）。
 - 结合 `LayoutManager` 实现列表、网格、瀑布流等布局。
- **示例：**

Code block

```
1 public class MyAdapter extends RecyclerView.Adapter<MyAdapter.ViewHolder> {
2     private List<String> mData;
3
4     public MyAdapter(List<String> data) { this.mData = data; }
5
6     static class ViewHolder extends RecyclerView.ViewHolder {
7         TextView tvItem;
8         ViewHolder(View itemView) {
9             super(itemView);
10            tvItem = itemView.findViewById(R.id.tv_item);
11        }
12    }
13
14    @Override
15    public ViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {
16        View view = LayoutInflater.from(parent.getContext())
```

```
17         .inflate(R.layout.item_layout, parent, false);
18         return new ViewHolder(view);
19     }
20
21     @Override
22     public void onBindViewHolder(ViewHolder holder, int position) {
23         holder.tvItem.setText(mData.get(position));
24     }
25
26     @Override
27     public int getItemCount() { return mData.size(); }
28 }
```

二、第三方适配器框架

原生适配器在复杂场景下（如多类型布局、分页加载、数据刷新）需要大量模板代码，第三方框架通过封装简化开发，提升效率。

1. BaseRecyclerViewAdapterHelper (BRVAH)

- **简介：**目前最流行的RecyclerView适配器框架，简化多布局、加载更多、拖拽排序等功能。
- **核心功能：**
 - 单布局/多布局快速实现。
 - 内置下拉刷新、上拉加载更多。
 - 支持侧滑删除、拖拽排序。
 - 数据观察者自动处理刷新。
- **使用示例：**

Code block

```
1  // 单布局适配器
2  public class MyAdapter extends BaseQuickAdapter<String, BaseViewHolder> {
3      public MyAdapter() {
4          super(R.layout.item_layout, getData());
5      }
6
7      @Override
8      protected void convert(BaseViewHolder helper, String item) {
9          helper.setText(R.id.tv_item, item); // 直接绑定数据
10     }
11 }
```

```

12
13 // 多布局适配器
14 public class MultipleAdapter extends
    BaseMultiItemQuickAdapter<MultiItemEntity, BaseViewHolder> {
15     public MultipleAdapter(List<MultiItemEntity> data) {
16         super(data);
17         addItemType(TYPE_TEXT, R.layout.item_text);
18         addItemType(TYPE_IMAGE, R.layout.item_image);
19     }
20
21     @Override
22     protected void convert(BaseViewHolder helper, MultiItemEntity item) {
23         switch (helper.getItemViewType()) {
24             case TYPE_TEXT:
25                 helper.setText(R.id.tv_text, ((TextItem)
item).getContent());
26                 break;
27             case TYPE_IMAGE:
28                 helper.setImageResource(R.id.iv_image, ((ImageItem)
item).getResId());
29                 break;
30         }
31     }
32 }

```

- 依赖:

Code block

```

1 implementation 'com.github.CymChad:BaseRecyclerViewAdapterHelper:3.0.7'

```

2. SmartRefreshLayout + SmartAdapter

- **简介:** SmartRefreshLayout专注于下拉刷新/上拉加载, SmartAdapter适配RecyclerView, 支持数据绑定和多类型。
- **特点:** 轻量级, 支持DataBinding和ViewBinding。
- **示例:**

Code block

```

1 SmartAdapter.create(data, R.layout.item_layout)
2     .bind({ holder, item ->
3         holder.setText(R.id.tv_item, item)
4     })

```

```
5         .attachTo(recyclerView);
```

3. Epoxy

- **简介：**由Airbnb开源，适用于复杂布局（如首页多模块卡片），通过注解生成ViewHolder，支持DiffUtil自动刷新。
- **核心优势：**
 - 声明式API，减少模板代码。
 - 自动处理视图复用和数据更新。
 - 支持嵌套RecyclerView。
- **使用示例：**

Code block

```
1  @EpoxyModelClass
2  public abstract class ItemModel extends
    EpoxyModelWithHolder<ItemModel.Holder> {
3      @EpoxyAttribute String text; // 数据属性
4
5      @Override
6      protected int getDefaultLayout() { return R.layout.item_layout; }
7
8      @Override
9      public void bind(Holder holder) {
10         holder.tvItem.setText(text);
11     }
12
13     static class Holder extends EpoxyHolder {
14         TextView tvItem;
15         @Override
16         protected void bindView(View itemView) {
17             tvItem = itemView.findViewById(R.id.tv_item);
18         }
19     }
20 }
21
22 // 组装数据
23 EpoxyController controller = new EpoxyController() {
24     @Override
25     protected void buildModels() {
26         for (String item : data) {
27             new ItemModel_()
28                 .text(item)
29                 .id(item)
```

```

30         .addTo(this);
31     }
32 }
33 };
34 recyclerView.setAdapter(controller.getAdapter());

```

- 依赖:

Code block

```

1  implementation 'com.airbnb.android:epoxy:4.6.3'
2  kapt 'com.airbnb.android:epoxy-processor:4.6.3'

```

4. DataBinding/ViewBinding + 原生适配器

- 简介: Android Jetpack组件, 通过XML绑定数据, 减少findViewById和手动赋值。
- 结合RecyclerView示例:

Code block

```

1  // Item布局 (item_layout.xml)
2  <layout>
3      <data>
4          <variable name="item" type="String" />
5      </data>
6      <TextView
7          android:id="@+id/tv_item"
8          android:text="@{item}" />
9  </layout>
10
11  // 适配器中使用ViewBinding
12  @Override
13  public void onBindViewHolder(ViewHolder holder, int position) {
14      holder.binding.setItem(data.get(position));
15      holder.binding.executePendingBindings();
16  }
17
18  static class ViewHolder extends RecyclerView.ViewHolder {
19      private ItemLayoutBinding binding;
20      ViewHolder(View itemView) {
21          super(itemView);
22          binding = ItemLayoutBinding.bind(itemView);
23      }
24  }

```

5. ListAdapter (Jetpack)

- **简介：**AndroidX提供的RecyclerView适配器，内置DiffUtil，自动计算数据差异并刷新。
- **特点：**适合数据频繁更新的场景，避免全量刷新。
- **示例：**

Code block

```
1  public class MyListAdapter extends ListAdapter<String,  
    MyListAdapter.ViewHolder> {  
2      protected MyListAdapter() {  
3          super(DIFF_CALLBACK);  
4      }  
5  
6      private static final DiffUtil.ItemCallback<String> DIFF_CALLBACK =  
7          new DiffUtil.ItemCallback<String>() {  
8              @Override  
9              public boolean areItemsTheSame(String oldItem, String newItem) {  
10                 return oldItem.equals(newItem);  
11             }  
12  
13             @Override  
14             public boolean areContentsTheSame(String oldItem, String  
15                 newItem) {  
16                 return oldItem.equals(newItem);  
17             }  
18  
19             // ... ViewHolder和绑定逻辑同原生适配器  
20     }
```

三、选型建议

1. **简单列表：**原生ArrayAdapter/SimpleAdapter或BRVAH单布局。
2. **复杂列表（多布局/加载更多）：**BRVAH（上手快、功能全）。
3. **高性能数据刷新：**ListAdapter (Jetpack) 或Epoxy（复杂布局）。
4. **MVVM架构：**结合DataBinding/ViewBinding + 原生适配器/ListAdapter。

第三方框架虽简化开发，但需权衡学习成本和项目依赖体积。对于简单场景，原生适配器已足够；复杂场景推荐BRVAH或Epoxy。

（注：文档部分内容可能由 AI 生成）